

Article

Lightweight Quantized XGBoost for Botnet Detection in Resource-Constrained IoT Networks

Mohammed Rauf Ali Khan ^{1,*}, Abdulaziz Y. Barnawi ^{1,2,*}, Adnan Munir ¹, Zainab Alsalman ¹ and Dario Marcelo Satan Sanunga ¹

¹ Department of Computer Engineering, King Fahd University of Petroleum and Minerals, Dhahran 31261, Saudi Arabia; g202110550@kfupm.edu.sa (A.M.); g202317050@kfupm.edu.sa (Z.A.); g202393010@kfupm.edu.sa (D.M.S.S.)

² IRC for Intelligent Secure Systems, King Fahd University of Petroleum and Minerals, Dhahran 31261, Saudi Arabia

* Correspondence: g202320050@kfupm.edu.sa (M.R.A.K.); barnawi@kfupm.edu.sa (A.Y.B.)

Abstract

The rapid expansion of IoT devices has introduced significant security challenges, with malware authors constantly evolving their techniques to exploit vulnerabilities in IoT networks. Despite this growing threat, progress in developing effective detection solutions remains limited. In this study, we present an ML-based framework for detecting and classifying network threats targeting IoT environments. Using the CTU-IoT-Malware-Capture 2023 dataset and the UNSW Bot-IoT dataset, we transformed the task into a structured multi-class classification problem to better reflect real-world detection challenges. Our primary contribution lies in demonstrating the effectiveness of post-training quantization on gradient-boosted models, specifically a Quantized XGB variant enhanced with histogram-based quantization. This approach significantly reduces model size and inference time without sacrificing accuracy. The proposed model achieved high classification accuracies of 99.93% and 99.99% on the two datasets, while the quantization step led to 1.42× and 3× improvements in inference speed, and reductions in model size by 3.61× and 2.71×, respectively, making it well-suited for deployment in resource-constrained IoT settings. This work demonstrates not only the effectiveness of gradient boosting in handling complex traffic data but also introduces an efficient optimization strategy for real-time IoT threat detection.

Keywords: IoT security; malware detection; machine learning; quantized XGBoost



Academic Editor: Amiya Nayak

Received: 23 September 2025

Revised: 3 November 2025

Accepted: 10 November 2025

Published: 18 November 2025

Citation: Rauf Ali Khan, M.; Barnawi, A.Y.; Munir, A.; Alsalman, Z.; Satan Sanunga, D.M. Lightweight Quantized XGBoost for Botnet Detection in Resource-Constrained IoT Networks. *IoT* **2025**, *6*, 70. <https://doi.org/10.3390/iot6040070>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

IoT applications are being majorly used to enhance daily life activities. From smart homes, smart cities, smart watches, agricultural and supply chain innovation, and smart lighting to electronic door locks and smart watches, humans rely on IoT networks to ease their day-to-day chores. Statista reports show that 17.08 billion IoT devices are being used in 2024 so far and they predict this number to become 30 billion by the end of 2024 [1]. The increase in usage will result in the rapid development of a variety and number of devices that will further lead to several security issues in IoT networks. Figure 1 illustrates an IoT network, highlighting the interconnected devices and communication paths that form the foundation of modern smart systems. This setup demonstrates how sensors, gateways, and cloud services collaborate to enable seamless data exchange and real-time monitoring.

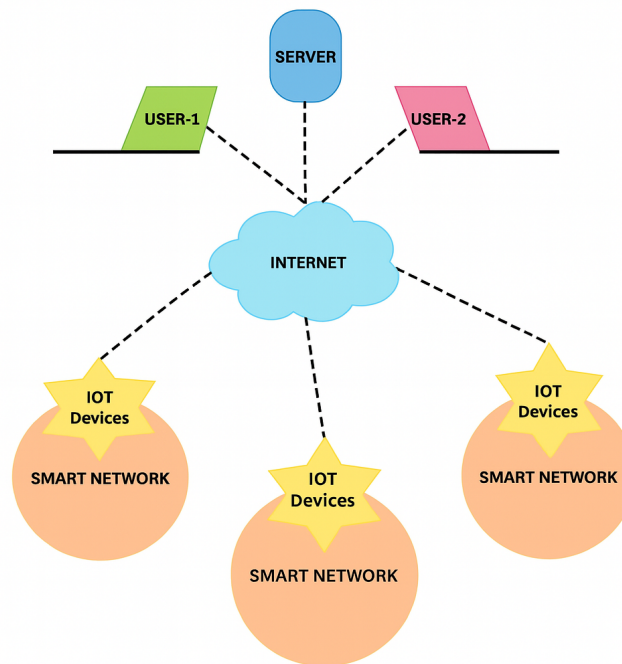


Figure 1. A general IoT network illustrating the interaction between sensors, edge devices, gateways, and cloud infrastructure. The diagram highlights how data flows from physical devices to centralized platforms for processing, analysis, and decision-making.

Botnet attacks are very common in IoT networks. Botnets, themselves, are compromised machines that are controlled by attackers to deliver payloads to other devices connected to the network as shown in Figure 2. With the increase in network devices, the rate of botnet attacks is also growing [2,3]. There were 4.2 billion botnet attacks that were reported by the end of 2023. Most of the reported attacks were in the DDoS category.

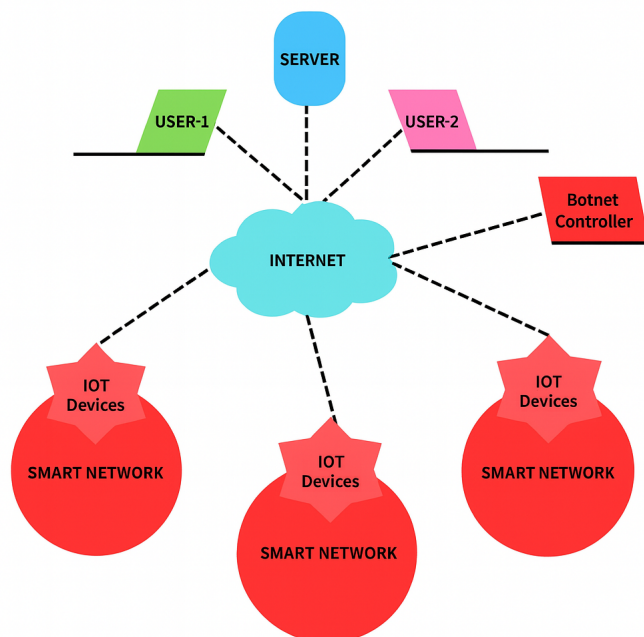


Figure 2. Representation of botnet controller interactions affecting three IoT networks. The diagram depicts a scenario where a malicious entity exerts unauthorized control over connected devices, highlighting security risks in IoT ecosystems.

ML and DL algorithms are widely used to write cyber-threat detection mechanisms [4,5]. One such idea of using machine and DL for classifying network traffic using packet capture data into web, email, social media applications, and direct traffic. This approach is also utilized to categorize network traffic into benign and malicious classes.

IoT networks are increasingly vulnerable to sophisticated botnet attacks and unauthorized access. Traditional security mechanisms often struggle to keep pace with the scale and complexity of modern IoT threats, particularly when dealing with high-dimensional, imbalanced traffic data. There is a need for intelligent, lightweight, and real-time threat detection solutions that can adapt to these dynamic environments.

Despite growing research efforts in IoT threat detection, several critical challenges persist. Many existing studies rely on highly imbalanced datasets, often favoring the infected class and fail to implement proper balancing techniques. This leads to biased models that struggle to generalize and accurately detect minority-class threats in real-world scenarios. Furthermore, performance evaluations are frequently conducted on limited or selectively chosen subsets of data, casting doubt on the reliability and applicability of the reported results. Compounding this issue, most solutions are not optimized for deployment on resource-constrained IoT devices, where memory and computational power are severely limited. DL models, while powerful, are often impractical in such environments due to their high resource demands. Additionally, the prevalent focus on binary classification in prior works limits the ability to identify and differentiate between specific types of attacks—an essential requirement for targeted threat mitigation. These limitations underscore the urgent need for lightweight, multi-class, and well-generalized models that are both accurate and feasible for real-time IoT security. Our study addresses these challenges by using ML techniques to improve the accuracy and efficiency of threat detection in IoT network traffic.

Key Contributions:

- We approach IoT botnet detection as a multi-class classification problem, allowing the identification of specific threat types rather than just detecting presence or absence of malicious traffic.
- We demonstrate the effectiveness of post-training quantization on gradient-boosted models for IoT security, achieving a significant reduction in resource requirements with minimal impact on performance. Our model achieves a high accuracy of 99.93% and 99.99%, outperforming several existing ML and DL methods.
- We provide a comparative analysis with recent state-of-the-art models to highlight the effectiveness and efficiency of our approach.

2. Related Work

A thorough grasp of the concepts, methods, and difficulties involved in identifying and classifying the many kinds of data that move across the networks is required for the background in network traffic classification. For network administration, security, and optimization, this sector is essential. These are the major elements of the network traffic categorization backdrop.

2.1. Background and Related Work

- Port-Based Classification

To differentiate between various services or protocols, the Internet-issued Numbers Authority issued port numbers to packets, which were then examined by early techniques of categorizing network traffic. System ports (0–1023), user ports (1024–49,151), and dynamic ports (49,152–65,535) are the three categories into which port numbers fall [6]. To classify packets, the port numbers of UDP and TCP packets that were mapped to IANA-defined ports had to be examined. Port 80, for instance, is asso-

ciated with the HTTP protocol [7]. The study by Moore and Papagiannis reported that, regardless of the number of packets analyzed, port-based identification methods achieved an accuracy of no more than 70% [6]. According to Madhukar and Williamson's assessment using private Internet traffic, the approach was unable to recognize between 30% and 70% of the data [8]. It was found that conventional ports made up just 30% of the dataset when they examined the efficacy of port-based techniques for P2P network traffic.

- **Protocol-Based Classification**

This is among the longstanding methods for categorizing network traffic, achieved by identifying the protocol identification in the packet header. While it offers a fast and straightforward solution, it is susceptible to manipulation or attacks [9].

- **Feature Extraction-Based Classification**

This approach includes the analysis of many packet parameters, such as timestamp and packet size, among others, with the main goal being the classification of network traffic. The manual selection and extraction of characteristics is necessary for the implementation of this strategy. It is crucial to remember that the manual component of the categorization process adds subjectivity, which might affect the objectivity and accuracy of the outcomes [10]. The process of identifying and extracting characteristics becomes a critical decision point, and the system's overall efficacy may be impacted by the subjective nature of these decisions.

- **ML-Based Classification**

The process of automatically identifying and classifying various forms of network traffic requires the use of models and algorithms in ML-based network traffic classification. When handling the growing complexity and diversity of network applications and protocols, this method is especially helpful. This method utilizes ML techniques such as neural networks and SVM to classify and recognize network traffic. These approaches offer a clear understanding of the categorization outcomes [11]. Given that the examination of classification results can reveal important aspects and patterns within traffic data, the interpretability of these results holds paramount importance in the traffic classification process.

- **DL-Based Classification**

This method categorizes and recognizes network traffic by employing advanced DL algorithms such as CNNs and RNNs. This approach stands out for its enhanced accuracy and flexibility, as it can automatically extract relevant characteristics from the data [12].

2.2. Literature Survey

Alissa et al. 2022 [13] discussed botnet detection in IoT using ML algorithms. The UNSW-15 dataset along with SMOTE, that was utilized by the researchers to effectively train the XGB, DT Classification, and LR algorithms. After applying an 80-20 train-test split to the dataset, an accuracy of 94% was achieved for DT Classification, 78% for LR, and 94% for XGB. The research highlights potential applications of Support Vector Machines, Random Forest Classification, and sophisticated DL methods including Residual Networks (ResNet-50) and LSTM networks. To improve cybersecurity and lessen possible dangers, the study emphasizes how important it is to include these detection techniques in the network applications' backends.

Araujo et al. [14] examined how ML approaches were applied to evaluate and derive insights from four different datasets—ISOT HTTP Botnet [15], CTU-13 [16], CICDDoS2019 [17], and BoT-IoT [18]. A key outcome of their effort was the development of a novel machine-learning pipeline called ANTE, which was painstakingly created for the thorough testing

of several algorithms. Their study's results showed an astounding 100% precision rate and an average detection accuracy of 99.06%. One significant obstacle that this investigation has to overcome, though, is the bias that is present in the datasets themselves, where there is a predisposition for the "clean" or "benign" class. Interestingly, the report doesn't go into detail on how to use any sample strategies to ensure a class balance in datasets.

Fok et al. examined the use of ML techniques to detect botnet traffic using support vector machines, DT, and RF classifiers [19]. The average recall percentages and FPRs for SVM, DT, and random forests are shown in the paper as 83.1%, 89.6%, and 84.9% respectively, offering insights into the efficacy of these classifiers. The results imply that botnet traffic identification accuracy may be improved, resulting in a notable decrease in FPRs. The authors specifically highlight the possibility of using DL techniques for these kinds of categorization issues in further studies. Utilizing a basic amount of dataset features such as flow size in bytes and packets, flow inter-arrival periods, and counts of illegal TCP flows is the main goal of the suggested technique. In addition to improving detection efficiency, this condensed feature set indicates room for future development in the field of ML-based automated botnet traffic identification.

Saied et al., 2023 [20], sought to improve botnet attack detection, mitigation, and classification accuracy in IoT environments. They used numerous ET learning algorithms to do this, and they used the N-BaIoT17 [21] dataset which is particularly made for IoT environments to perform their research. AdaBoost, Gradient Descent Boosting, DT, Random Forest, Bagging Meta Classifier, and XGB are the six tree-based methods that were evaluated, implemented, and tested as part of this research. After a thorough evaluation, the authors concluded that the Random Forest algorithm performed the best in both intrusion detection and multi-class categorization. The program attained a remarkable 99.9991% accuracy rate. It also showed respectable test and training times, with timings of 4.33 s and 1249.52 s, respectively. The results highlight how well the Random Forest algorithm handles the intricacies involved in identifying and categorizing botnet attacks on IoT devices, which adds important context to the current efforts to strengthen IoT ecosystem security.

An extensive survey on network traffic classification was carried out by Sheikh and Peng [22]. By analyzing current methods, their study offered a comprehensive analysis of the several approaches, strategies, and datasets used in the traffic categorization process. They specifically explored ML techniques for traffic categorization, providing an extensive overview of the latest research efforts in this field. Four main categories of classification strategies were identified in the paper including behavior-based, statistical-based, payload-based, and port-based classifications. They also covered parameters that are necessary for researchers to assess the effectiveness and performance of traffic categorization techniques.

Koroniotis et al. proposed a brand-new cloud-based approach that uses a ML algorithm to identify and categorize botnet activity [23]. The three main modules of the system architecture were detection or classification, filtering, and feature extraction. Using a bagging classifier and a DT ML method, the system was trained. Using the Botnet-Detection dataset [24] which was taken from the broader ISCX UNB dataset [25] and contained trojan, clean data, and random bots, the system was trained. The study's conclusions showed how well the system identified botnets and detected unusual flows. The scientists did, however, identify several efficiency problems with the system, such as the size of the cloud's constraints and the length of training sessions. Notwithstanding these obstacles, by utilizing cloud-based solutions and ML techniques, the research advances botnet detection systems. The results provide insightful information on the system's strengths and weaknesses, opening the door for further improvements and optimizations in cloud-based botnet traffic detection in the future.

Azab et al. performed a thorough analysis of the techniques, datasets, and machine-learning algorithms used in network traffic categorization [9]. They provided a thorough analysis of both ML and DL approaches for categorization, outlining the benefits, limitations, implementations, and limits of each strategy. Numerous algorithms, including supervised, unsupervised, and semi-supervised ML techniques, were examined in the study. The results of the study show that no one solution can achieve perfect performance in terms of speed, accuracy, and early-stage detection. The authors recommended using multilayer classification models as a solution to the drawbacks of current methods.

Rachmawati et al. conducted a thorough investigation of the widely used DL methods in traffic categorization problems [26]. They did a thorough evaluation of current contributions in the area and developed a complete framework covering many elements of DL-based approaches. They categorized the contributions based on data preparation, pre-processing, model input design, and model architecture. The paper also meticulously outlined the challenges associated with deploying DL in traffic classification, shedding light on its inherent limitations. In envisioning future research directions, the authors put forth a proposal for the development of a deep-learning model specifically tailored for the classification of encrypted traffic. This suggested avenue for exploration could potentially address current gaps and contribute to advancing traffic classification methodologies in a rapidly evolving technological landscape.

Esmailyfard et al. used the UNSW Bot-IoT dataset to detect IoT botnet attacks [27]. A stacked ensemble model combining MLP and RF achieved the best accuracy of 99.3%. To make the system lightweight, the authors applied lasso-based feature selection and LR stacking. This reduced resource use, requiring 36% less CPU and 38% less memory, making it suitable for IoT devices. These findings motivate us to design ML models for fulfilling the constraints of resource-limited devices.

In recent years, there has been a growing trend towards applying XAI in botnet detection and classification for IoT security [28]. While ML-based methods have shown strong performance in identifying botnet activities, their increasing complexity raises concerns about transparency and trust. XAI techniques such as rule extraction, LIME, and SHAP are now being integrated to make these models more interpretable, enabling better trust, early attack detection, and stronger defense strategies. This shift reflects a broader movement in cybersecurity research toward building transparent, and trustworthy AI-driven systems for protecting IoT ecosystems.

Rupanetti and Kaabouch, in their work on applying machine learning for botnet attack detection, utilized the IoT-23 dataset and trained three models using a fixed train–test split [29]. In their results, the RF model achieved an accuracy of 99.0%. However, their approach primarily focused on evaluating model performance without addressing model optimization or deployment efficiency, which limits its applicability to real-time IoT environments. Ibrahim et al. discussed several other approaches for detecting IoT botnets based on DNS traffic analysis and host-based anomaly detection [30].

Table 1 provides the list of all datasets cited in this study.

Table 2 provides a comparative analysis of our study's results against several state-of-the-art approaches in IoT threat detection. It outlines the algorithms or models employed in each study, along with their respective datasets and evaluation metrics. Notably, hybrid models such as the combination of XGB, SVM, RF, ANN, and RNN achieved the highest accuracy of 99.996% on the N-BaIoT dataset, as demonstrated by Rawat et al. [31]. Other studies, including those utilizing DT, NB, and Hidden Markov Models, reported varying levels of accuracy based on dataset characteristics. In our study, we employed XGB with SMOTE for class balancing and achieved an accuracy of 99.93% and 99.99%, demonstrating its effectiveness in handling imbalanced data. The table highlights the diversity in method-

ologies and datasets used, emphasizing the significance of model selection in achieving optimal detection performance.

Table 1. List of datasets cited in this study.

Reference	Dataset
[15]	ISOT-HTTP Botnet
[16]	CTU-13
[17]	CIC-DDoS2019
[18]	UNSW Bot-IoT
[21]	N-BaIoT17
[24]	Botnet-Detection
[25]	ISCX UNB
[32]	UNSW-NB15
[33]	CIC-IDS2017
[34]	BoT IoT
[35]	ICS-Flow
[36]	CIC-IoT2023
[37]	ToN-IoT
[38]	N-BaIoT25

Table 2. Comparison of our approach with state-of-the-art approaches.

State-of-the-Art Study	Year	ML/DL Model	Dataset Description	Accuracy
Rawajbeh et al. [35]	2025	Adaptive Hoeffding Tree	ToN-IoT, Bot-IoT	96.40%
Pallakonda et al. [39]	2025	DT	ICS-Flow	99.81%
Ye et al. [40]	2025	XGB stacked with RF	CICIoT2023	95.9%
Nuha et al. [41]	2025	KNN	Non-Public Simulation Data	99%
Ali et al. [42]	2025	Stacked KNN, SVM, RF, DT and MLP	UNSW-NB15	97.94%
Mohan et al. [43]	2025	BiGRU	UNSW-NB15	99.22%
Kayyidavazhiyil [44]	2025	Ensemble of BiGRU, LSTM and SIBMO	TON-IoT	93%
Saied et al. [18]	2024	Histogram Gradient Boosting	N-BaIoT	99.97%
Tikekar et al. [45]	2024	NB	CTU-13 Dataset	90.62%
Hostiadi et al. [46]	2024	DT	NCC Dataset	99.03%
Rawat et al. [31]	2024	Hybrid of XGB, SVM, RF, ANN, RNN		99.996%
Mannikar and Troia [47]	2024	Hidden Markov Model	CTU-13 Dataset	83.19%
Bojarajulu and Tanwar [48]	2024	Customized CNN	TON_IoT, UNSW-NB15	88.42%
Saif et al. [49]	2023	Random Forest	N-BaIoT	99%
Chaganti et al. [50]	2023	LSTM	Simulated Data	97.1%
Sharma et al. [51]	2023	DNN	UNSW-NB15	91%
Santhadevi and Janet [52]	2023	LSTM	UNSW-NB15, UNSW_BOT_IoT	97.97%
Cam and Trung [53]	2023	Decision Tree	CICIDS 2017	99.90%
Our Approach	2025	Quantized XGB	CTU-IoT-Malware-Capture 2023 Bot-IoT Dataset 2019	99.93% & 99.99%

2.3. Summarized Problem Statement

The datasets utilized in the surveyed works predominantly exhibit an imbalance toward the infected class. However, the authors did not implement any data sampling or balancing techniques, leading to bias, overfitting, poor generalization, and misclassification of minority classes. Furthermore, the evaluation of model performance was conducted on a specific subset of the dataset, raising concerns regarding the reliability of the reported

results. The high accuracy, precision, and recall values observed in these studies are likely influenced by data imbalance and the selection of biased training and testing data, thereby limiting the models' real-world applicability.

Additionally, while several models have been proposed, their practical deployment on resource-constrained IoT devices remains a challenge due to limited memory availability, often in the KB range. This constraint makes it infeasible for DL models and even some ML algorithms to run efficiently on IoT endpoints. Moreover, the existing works primarily frame the problem as a binary classification task, which restricts the scope to mere detection and blocking of threats. However, effective cybersecurity measures require the capability to identify specific threat classes, enabling automated responses and appropriate mitigation strategies. Addressing these limitations is crucial for developing practical, scalable, and effective IoT security solutions.

3. Proposed Histogram Quantization in XGB

Gradient-boosted algorithms such as DT and XGB rely on recursive feature splitting to minimize loss functions. During training, these algorithms evaluate potential thresholds for each feature to find optimal splits that yield the highest information gain. However, this approach can become computationally expensive for large-scale datasets or those containing high-cardinality features (i.e., features with many unique values).

To address this challenge, histogram-based quantization is employed to discretize continuous or high-cardinality features into a fixed number of bins (or buckets). Instead of computing gradient statistics for every unique feature value, the model computes them for each bin, thereby reducing computational cost and memory overhead while retaining accuracy. This method is particularly effective for IoT and edge-computing environments, as it provides high computational efficiency and lower memory usage.

Example Feature: "Proto"

Consider the feature "proto", which represents the network communication protocol (e.g., TCP, UDP, ICMP). During quantization, these protocol types are treated as categorical inputs that are mapped into histogram bins.

Step 1: Initialization.

The unique values of the "proto" feature are first sorted and divided into k predefined bins, forming the histogram $\mathcal{B}$:

$$\mathcal{B} = \{b_1, b_2, \dots, b_k\}, \quad \text{where } k = \text{number of bins.} \quad (1)$$

Each feature value $x \in \text{proto}$ is assigned to one of these bins $b_j \in \mathcal{B}$. In this study, the number of bins (k) is empirically determined by the XGB quantizer, typically between 256 and 1024, depending on the feature distribution. This ensures an adequate trade-off between granularity and computational efficiency.

Step 2: Histogram Construction.

For each bin, the algorithm aggregates gradient information from all samples that fall within it. Two key statistics are collected:

- **First-order gradient (g_i):** The derivative of the loss function with respect to the model's prediction for each sample, representing the direction and magnitude of the error.
- **Second-order gradient (h_i) (Hessian):** The second derivative of the loss function, representing the curvature or confidence in the first-order gradient.

These gradients are accumulated for each bin as follows:

$$G_j = \sum_{x_i \in b_j} g_i, \quad H_j = \sum_{x_i \in b_j} h_i \quad (2)$$

Here, G_j and H_j denote the total first- and second-order gradient sums for bin b_j , which are later used to evaluate potential split gains.

Step 3: Determining Optimal Splits.

The algorithm evaluates all possible split points at the boundaries of the histogram bins to find the one that maximizes the information gain:

$$\text{Gain} = \frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \quad (3)$$

where G_L, H_L correspond to the gradient and Hessian sums for the left child, and G_R, H_R for the right child. The regularization term λ prevents overfitting by penalizing overly complex splits.

Step 4: Splitting Nodes.

If the protocol feature value maps to a bin index $j \leq j^*$, where j^* represents the optimal split boundary, the sample is assigned to the left child; otherwise, it goes to the right child:

$$x \in \text{proto} \Rightarrow \begin{cases} \text{left child,} & \text{if } \text{bin}(x) \leq j^* \\ \text{right child,} & \text{otherwise.} \end{cases} \quad (4)$$

Step 5: Tree Growth and Stopping Criteria.

This histogram building and splitting process is repeated recursively for all features until a stopping condition is met—such as reaching the maximum tree depth or when the gain improvement becomes negligible:

$$\text{Tree stops when: } \text{depth} \geq D_{\max} \quad \text{or} \quad \text{Gain} \leq \epsilon \quad (5)$$

This quantization strategy enables the model to efficiently handle high-dimensional IoT traffic data while preserving predictive accuracy. By converting feature values into histogram bins and computing gradient statistics per bin rather than per individual value, the training complexity is significantly reduced. The first-order gradients represent the direction of loss reduction, while the second-order gradients (Hessians) reflect the curvature, helping the model determine the optimal step size for updating decision thresholds.

The number of initial categories (bins) depends on feature distribution. Here, continuous features may be divided into hundreds of bins, while categorical features (like proto) may only require a few. In our case, the XGB quantizer dynamically adapts the bin count based on feature variance.

4. Experimentation and Results

In this section, we will describe the dataset and methods we used for network classification. As ML does not take strings and IP addresses as input, we will pre-process data to convert the non-numeric types to numeric values. The overall flow diagram of this work is shown in Figure 3.

- We have used various botnet variants to train multi-class classifiers with higher accuracies. The classes included Port Scan, DDoS, C2, Mirai, Heartbeat, Torii, OS Fingerprinting, Service Scan, and Data Ex-filtration attack traffic.
- We have used the SMOTE for balancing the classes in the dataset.

- We have evaluated model performances by two techniques: selecting random test sets and 10-fold CV.
- XGB was found to be efficient in our case.
- The model was dumped as a lightweight ML entity.
- This will consume less memory and work on low-memory IoT platforms.

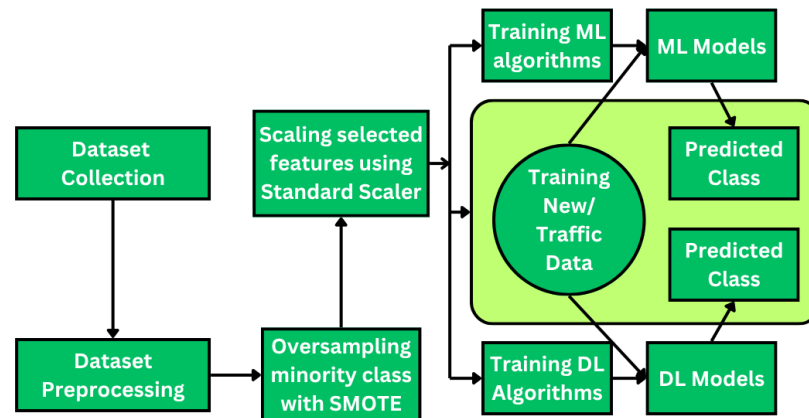


Figure 3. Proposed methodology for training IoT botnet detection model, the pipeline begins with dataset collection and proceeds through preprocessing, oversampling using SMOTE, and feature scaling. The processed data is then used to train both machine learning and deep learning algorithms, leading to the prediction of traffic classes.

4.1. The Dataset

In our study, we have used the “Malware Detection in Network Traffic” dataset that was published by Stratosphere Labs in 2023 [24] and “Bot-IoT dataset” that was published by UNSW in 2019 [18]. The first dataset contains traffic capture for several malicious activities that can be observed in IoT network traffic. The following list describes the captured malicious traffic categories.

- Command and Control: This class signifies that the device is infected and has a connection established with the C2 server.
- DDoS: This type of traffic is to signify a huge volume of traffic flow towards a single IP address.
- File Download via C2: It signifies that there is a download on the device, and it is occurring on some unusual port numbers.
- Heartbeat: This traffic is usually around 1KB in size and is sent by the C2 server to check if the infected victim is still connected or not.
- Port Scan: This type of traffic is usually intended to collect potential information about the device that can be useful to create subsequent attacks.
- Specific Botnet Traffic: The dataset also contains traffic captures related to Mirai and Torii botnets.
- Benign: To specify non-malicious traffic.

The second dataset has the following six traffic categories.

- Service Scan: This type of traffic reflects attempts to identify running services on a device, often to find exploitable entry points.
- OS Fingerprinting: This involves probing techniques aimed at uncovering the target device’s operating system for tailored attacks.
- Keylogging: This class includes traffic patterns that suggest user keystrokes are being secretly captured and transmitted.
- DoS: It represents attack traffic designed to overwhelm and crash a service or device using repeated requests.

- Data Exfiltration: This signifies attempts to sneak sensitive data out of a network, usually to an attacker-controlled location.
- Normal: Indicates regular, harmless network activity that poses no security threat.

The datasets had individual files for each of the attacks discussed and had several tuples for each of the classes. Figure 4 represents the number of samples available for each threat class.

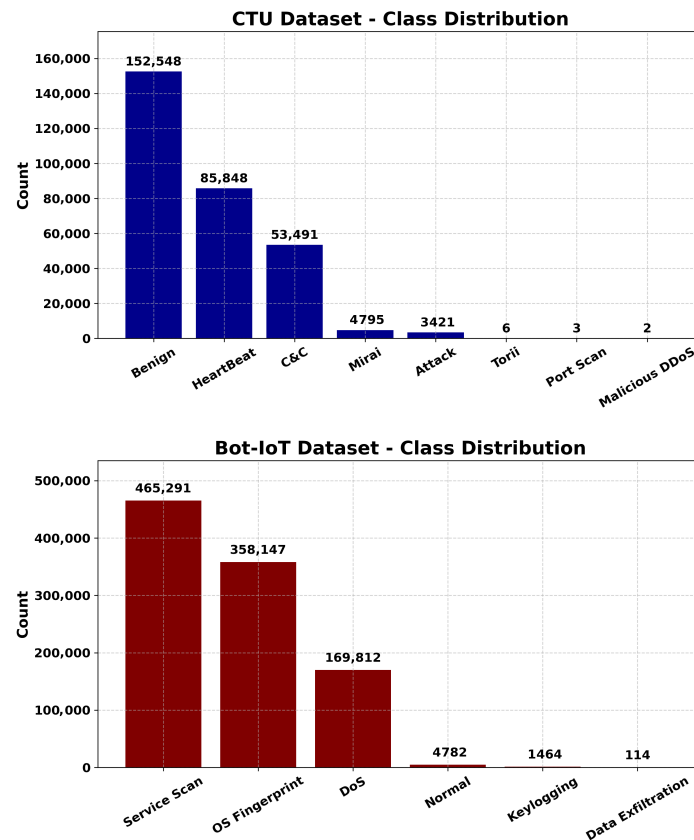


Figure 4. Class-wise sample distribution in the datasets.

The proposed architecture contains a lightweight machine-learning model running on-site as shown in Figure 5. This supports the presence of a ML-based detection mechanism at the end of an IoT device.

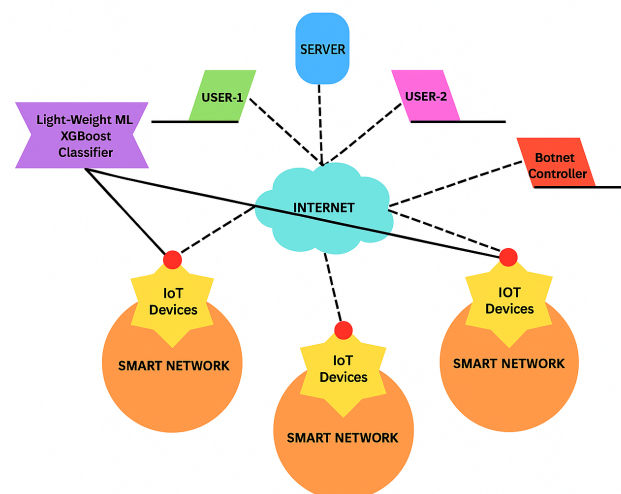


Figure 5. Proposed Light-Weight ML-Based IoT Architecture for Detecting Botnet Traffic.

4.2. Dataset Pre-Processing

Although there are several rows in the datasets, we initially observed accuracies around 99.99% to 100% when we first trained the model. Then, we found that each dataset had a class imbalance problem that led to overfitting. To address this, we used the SMOTE technique on the training data alone to eliminate class imbalance and produce lower accuracies but better generalization. SMOTE takes one data value at a time, computes its distance to the nearest neighbor, and multiplies it by a number between 0 and 1. These newly computed values are used to create synthetic samples for the minority classes, scaling all classes up to match the majority class count. A figure illustrating the class distributions before and after applying SMOTE has been included to strengthen this explanation.

We also removed the ‘.’ from the columns containing IP addresses, mapping them to numeric values. The ‘LabelEncoder’ was used to convert other categorical text columns into numerical form, replacing text entries with integers based on alphabetical order.

The Standard Scaling mechanism was then used to normalize the dataset parameters for faster training and improved accuracy. This procedure converts all feature values to floating-point numbers within the range of zero to one.

4.3. Experimentation Setup

The experiments were conducted using a dataset containing network traffic features extracted from IoT devices, with the objective of classifying botnet and benign traffic. The system configurations include a 16 GB RAM, core i5 13 generation processor on a Windows 11 machine. The dataset was preprocessed by standardizing the numerical features using the StandardScaler from scikit-learn, ensuring that all input features had a uniform scale. To address the class imbalance, present in the dataset, the SMOTE was applied, generating synthetic samples for the minority class to enhance classification performance. A 10-fold CV strategy was employed to ensure robustness and generalization of the models. Various ML algorithms, including LR, NB, DT, KNN, and XGB, were trained and evaluated using this setup. Additionally, DL models such as ANN, LSTM, and GRU were implemented for multi-class classification.

For the ML models, the LR classifier was set to use the ‘lbfgs’ solver with a maximum of 10 iterations and an ‘ovr’ multi-class strategy. The KNN classifier used 20 neighbors, while the DT classifier operated with its default settings. XGB, which yielded the highest accuracy, was configured using its default hyperparameters. No additional hyperparameter tuning was performed, as the objective of this study was to evaluate the baseline effectiveness of quantization on model efficiency and inference performance rather than to optimize for peak accuracy. This approach ensures that the reported improvements in speed and model compactness are attributable solely to the quantization process, independent of any tuning-related performance gains. The DL models were built using Keras Sequential API, with two hidden layers, each having 16 neurons and ReLU activation, followed by a softmax output layer for multi-class classification. The ANN, LSTM and GRU models were trained for five hundred epochs using a batch size of 100 and Adam optimizer, with sparse categorical cross-entropy loss to handle integer-encoded class labels. The evaluation was performed using standard classification metrics, including accuracy, precision, recall, and F1-score, ensuring a comprehensive assessment of model performance.

All experiments were implemented in Python 3.11, leveraging widely used ML and data analysis libraries. The core implementation utilized ‘scikit-learn’ for preprocessing, feature scaling, SMOTE oversampling, and evaluation metrics. The ‘imbalanced-learn’ package was employed for handling class imbalance, while ‘XGBoost’ library was used for implementing gradient-boosted models and quantization-based optimization. DL experiments were conducted using ‘TensorFlow 2.15’ and ‘Keras 2.14.0’ for model construction,

training, and evaluation. Additional libraries such as ‘NumPy 1.24.4’, ‘Pandas 2.0.3’, and ‘Matplotlib 3.7.2’ were used for numerical computation, dataset management, and result visualization, respectively. This configuration ensured a reproducible and efficient experimental environment for evaluating both traditional ML and DL approaches in IoT botnet detection.

4.4. Results and Discussion

This section presents the results obtained from evaluation of various ML and DL algorithms. A set of performance metrics, including accuracy, precision, recall, F1-score, and FPR, was computed to provide a broad assessment of each model’s effectiveness in classifying network traffic.

4.4.1. Evaluation Metrics

The confusion matrix provides a classification report for a model. It specifies the number of correct classifications—True Positives and True Negatives—and the number of misclassifications—False Positives and False Negatives.

Accuracy:

Accuracy measures the percentage of test data that has been correctly classified as shown in Equation (6).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (6)$$

Precision:

Precision defines the ratio of correctly predicted positive observations to the total predicted positives as shown in Equation (7).

$$\text{Precision} = \frac{TP}{TP + FP} \quad (7)$$

Recall:

Recall measures the ratio of correctly predicted positive observations to all actual positives as shown in Equation (8).

$$\text{Recall} = \frac{TP}{TP + FN} \quad (8)$$

F1-Score:

The F1-score is the harmonic mean of precision and recall as shown in Equation (9).

$$\text{F1 Score} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (9)$$

Confusion Matrix Visualization:

The confusion matrix illustrates the count of correct and incorrect predictions made by the model. It helps in identifying which classes are correctly predicted and reveals any model bias toward a particular class.

ROC-AUC:

The ROC-AUC is a graphical representation of a classifier’s performance at various threshold levels. The curve plots the FPR on the x-axis and the TPR on the y-axis. The closer the area under the curve is to 1, the better the classifier performs.

4.4.2. Results

Five different ML algorithms were used to train and test models for solving the multi-class botnet traffic classification problem. Table 3 shows the metrics that we obtained by training ML models and testing with the aggregated dataset available for various botnet traffic types.

Table 3. ML performance evaluation for the CTU-IoT Botnet Dataset.

Metric	LR	NB	DT	kNN	XGB
Accuracy	85.9550	99.4196	99.9208	99.6218	99.9335
Precision	85.8342	99.4210	99.9220	99.6203	99.9337
Recall	85.9461	99.4185	99.9222	99.6178	99.9335
F1 Score	85.6648	99.4165	99.9222	99.6170	99.9335
AUC	0.9828	0.9990	0.9991	0.9996	0.9999

In Table 3, we observe that XGB outperforms the other four ML algorithms in aspects of all the considered evaluation metrics. In addition to these five ML algorithms, we also trained DL models and tested them using the 10-fold CV. We observed that although the GRU have the highest accuracy among all the trained DL algorithms, their accuracy is still less than the values we obtained for XGB. Table 4 shows the performance evaluation of the DL algorithms.

Table 4. DL performance evaluation for the CTU-IoT Botnet Dataset.

Metric	ANN	LSTM	GRU
Accuracy	99.45	99.83	99.86
Precision	99.80	99.90	99.91
Recall	99.80	99.90	99.91
F1 Score	90.40	92.70	85.65
AUC	0.9582	0.9684	0.9688

In Table 5, we note that XGB consistently delivers superior performance across all evaluated metrics compared to the other four ML algorithms. While DT and KNN also perform reasonably well, XGB stands out with near-perfect accuracy and precision. We further extended our evaluation by training DL models and validating them through 10-fold CV. As shown in Table 6, GRU achieves the best performance among the DL models, slightly surpassing LSTM and ANN. However, even the GRU model falls just short of the results obtained by XGB, reaffirming the strength of gradient boosting for this task.

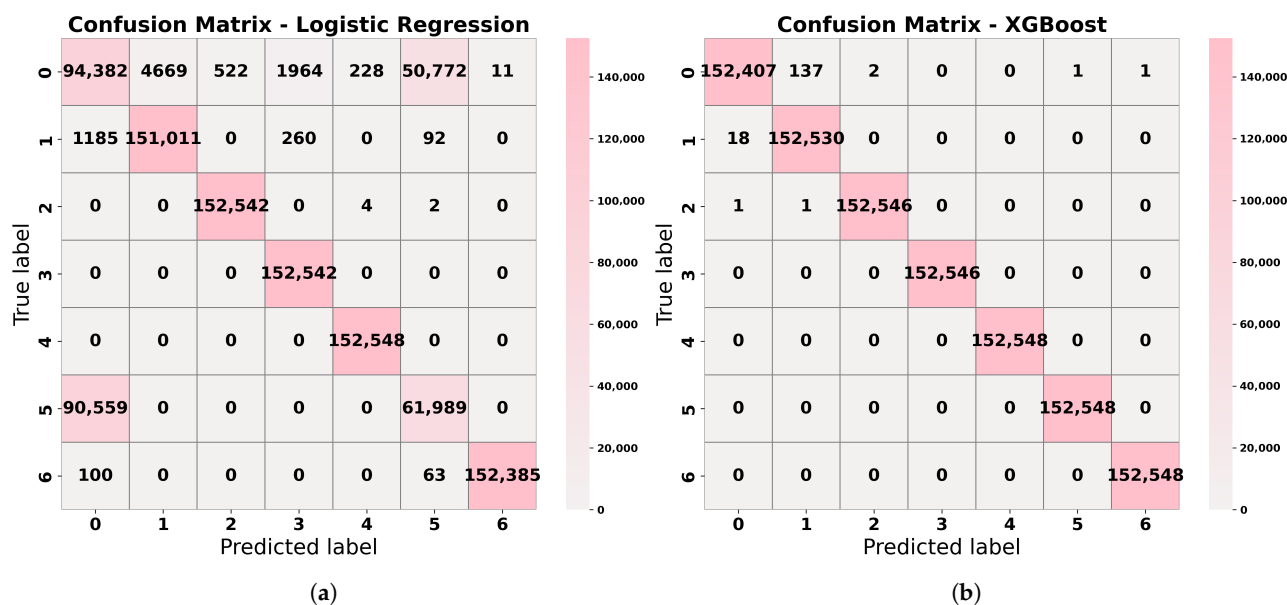
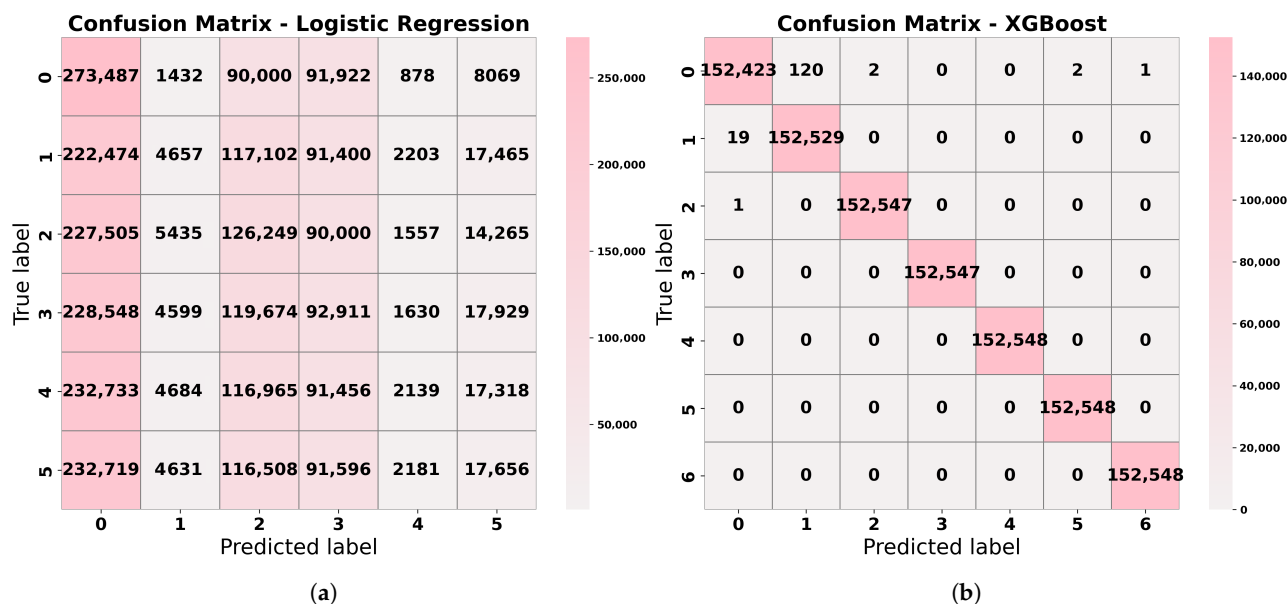
Table 5. ML performance evaluation for the Bot-IoT Dataset.

Metric	LR	NB	DT	kNN	XGB
Accuracy	18.52	18.13	67.07	99.79	99.99
Precision	18.66	19.91	67.17	99.79	99.99
Recall	18.52	18.13	67.08	99.79	99.99
F1 Score	13.04	8.83	67.09	99.79	99.99
AUC	0.5181	0.5190	0.8404	0.9997	0.9999

Following the application of SMOTE, the ML models, including XGB and KNN, demonstrated effective performance with fewer misclassifications. However, LR did not exhibit satisfactory performance even after employing SMOTE. Specifically, the LR classifier misclassified many test samples as shown in Figures 6 and 7.

Table 6. DL performance evaluation for the Bot-IoT Dataset.

Metric	ANN	LSTM	GRU
Accuracy	88.18	98.95	99.91
Precision	88.30	98.95	99.91
Recall	88.18	98.95	99.91
F1 Score	88.08	98.94	99.90
AUC	0.9916	0.9988	0.9997

**Figure 6.** Confusion Matrices for the CTU-IoT Botnet Dataset: (a) LR. (b) XGB.**Figure 7.** Confusion Matrices for the UNSW Bot-IoT Dataset: (a) LR. (b) XGB.

The ROC-AUC curves illustrate each model's ability to distinguish between multiple attack classes. For both datasets, LR shows weaker discrimination, with lower AUC scores and flatter curves closer to the diagonal, indicating limited predictive power. In contrast, XGB consistently achieves near-perfect AUC scores, with sharply rising curves approaching the top-left corner of the plot. This highlights XGB's superior ability to

separate classes accurately in both CTU-IoT and Bot-IoT datasets. These results reinforce the effectiveness of gradient boosting in complex network traffic classification tasks as shown in Figures 8 and 9.

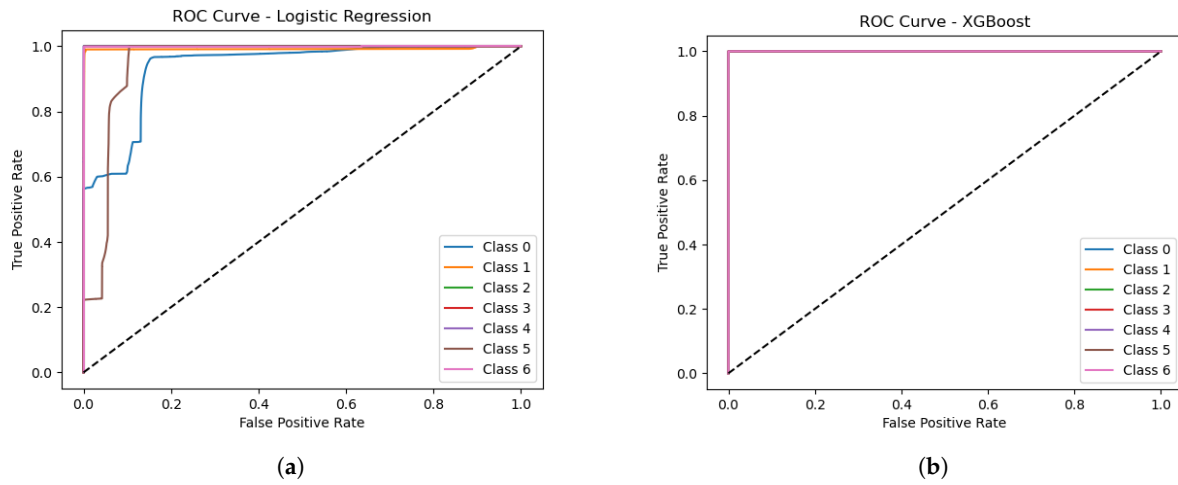


Figure 8. ROC-AUC Curves for the CTU-IoT Botnet Dataset: (a) LR. (b) XGB. (Dashed diagonal line denotes random chance; curves above it indicate meaningful predictive power.)

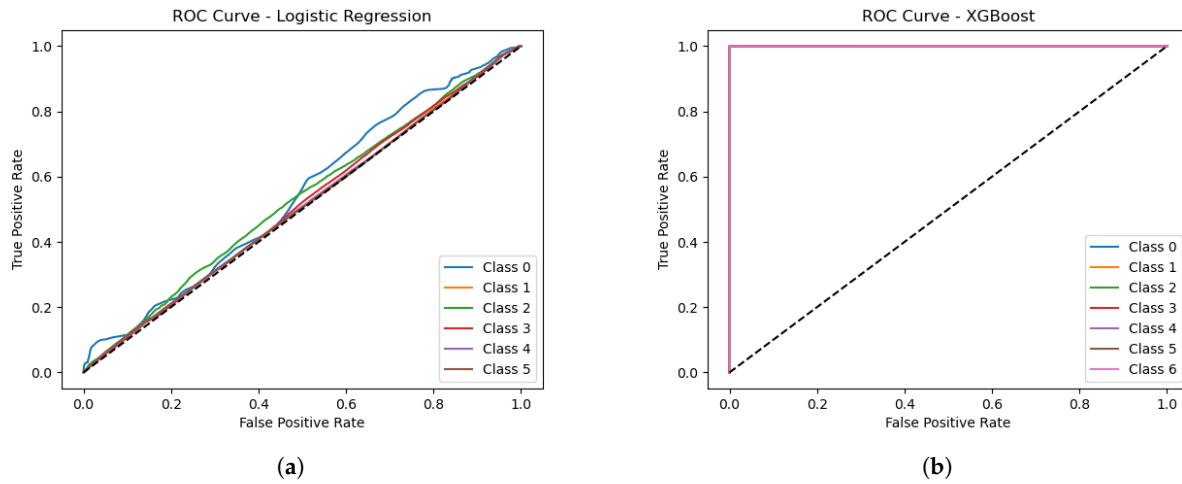


Figure 9. ROC-AUC Curves for the UNSW Bot-IoT Dataset: (a) LR. (b) XGB. (Dashed diagonal line denotes random chance; curves above it indicate meaningful predictive power.)

4.4.3. Discussion: Quantized XGB

The quantized versions of the XGB models were evaluated to assess their efficiency in real-time IoT scenarios. As shown in Table 7, quantization led to a substantial reduction in both model size and inference time, while maintaining near-identical classification performance. For the BOT-IoT dataset, the quantized model achieved the same accuracy of 99.99% as the full model, while reducing inference time from 0.0604 s to 0.0201 s—an improvement of approximately 3× in speed—and compressing the model size from 1128 KB to 305 KB (a 3.7× reduction). Similarly, for the CTU dataset, quantization improved inference speed by 1.42× (from 0.0151 s to 0.0106 s) and reduced model size from 991 KB to 365 KB, representing a 2.7× size reduction, with only a marginal drop in accuracy from 99.91% to 99.72%. These results highlight the advantage of using quantized XGB models for deployment in resource-constrained environments, where memory and latency are critical limitations.

Table 7. Comparison of Normal and Quantized XGB Models.

Metric	BOT-IoT (Normal)	BOT-IoT (Quantized)	CTU-IoT (Normal)	CTU-IoT (Quantized)
Accuracy (%)	99.99	99.99	99.91	99.72
Inference Time (s)	0.0604	0.0201	0.0151	0.0106
Model Size (KB)	1128	305	991	365

Among all evaluated algorithms, XGB consistently delivered the highest accuracy across both datasets. It outperformed traditional ML classifiers such as LR, NB, DT, and KNN, which often suffer from oversimplified assumptions or overfitting in imbalanced or high-dimensional settings. While DL models like ANN, LSTM, and GRU showed competitive performance, they are significantly more resource-intensive and require longer training and inference times, making them less suitable for real-time IoT intrusion detection. In contrast, XGB combines robustness, efficiency, and high predictive power. When paired with histogram-based quantization, it becomes even more viable for edge-level deployment, offering substantial gains in speed and model compression without sacrificing detection quality.

5. Conclusions

In this study, we developed an efficient XGB classifier for detecting malicious traffic in IoT networks. The model was trained and evaluated using the CTU-IoT and Bot-IoT datasets, both of which contain diverse attack patterns, including DDoS, port scanning, heartbeat anomalies, and botnet families such as Mirai and Torii. As shown in Tables 3 and 5, the proposed XGB model consistently achieved near-perfect classification performance, reaching 99.99% accuracy, precision, recall, and F1 score on the Bot-IoT dataset, and above 99.93% across all metrics on the CTU-IoT dataset. These results surpass traditional ML and DL methods such as LR, NB, DT, KNN, ANN, LSTM, and GRU, confirming the model's reliability in distinguishing benign and malicious network traffic.

Quantization further improved computational efficiency without sacrificing accuracy. As presented in Table 7, the quantized XGB model achieved up to $3\times$ reduction in inference time (from 0.0604 s to 0.0201 s on Bot-IoT) and a $3.7\times$ decrease in model size (from 1128 KB to 305 KB). Similar gains were observed for the CTU-IoT dataset. These improvements validate that the quantized model maintains detection accuracy while offering substantial savings in memory and processing requirements, making it highly practical for real-time deployment on resource-limited IoT devices.

Beyond its empirical performance, the proposed approach meets the operational needs of IoT environments, where computational resources and memory are often constrained. By minimizing overhead, the model supports continuous, on-device threat monitoring without disrupting normal network functions. Future work will explore adaptive techniques such as few-shot and one-shot learning to enhance detection under limited labeled data. Further optimization through pruning, knowledge distillation, and intermediate representation learning could also reduce computational load while maintaining accuracy. These directions may pave the way for even more efficient and scalable threat detection solutions tailored to next-generation IoT ecosystems.

Author Contributions: Conceptualization, A.Y.B., A.M., D.M.S.S., M.R.A.K., and Z.A.; Methodology, A.M., D.M.S.S., M.R.A.K., and Z.A.; Software, M.R.A.K.; Validation, A.Y.B., A.M., D.M.S.S., M.R.A.K., and Z.A.; Formal Analysis, A.M. and D.M.S.S.; Investigation, M.R.A.K. and Z.A.; Resources, Z.A.; Data Curation, M.R.A.K.; Writing-Original draft preparation, A.M., D.M.S.S., M.R.A.K., and Z.A.; Writing-Review and Editing, A.Y.B., A.M., D.M.S.S., M.R.A.K., and Z.A.; Visualization, M.R.A.K.; Supervision, A.Y.B.; Project Administration, A.Y.B., A.M.; Funding Acquisition, A.Y.B.; Revision, A.Y.B., M.R.A.K. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The datasets used in this study are publicly available. The Malware Detection in Network Traffic dataset is available at <https://www.stratosphereips.org/datasets-iot23> (accessed on 22 September 2025), and the Bot-IoT dataset is available at <https://doi.org/10.1109/MITP.2024.3466509> (accessed on 22 September 2025).

Acknowledgments: The authors acknowledge the support from the Department of Computer Engineering and the Interdisciplinary Research Center for Intelligent Secure Systems and the Computer Engineering Department at KFUPM. During the preparation of this manuscript, the authors used ChatGPT (OpenAI, GPT-4, June 2025 version) for assistance with language refinement, code explanation, and formatting of LaTeX content. The authors have reviewed and edited all generated content and take full responsibility for the integrity and accuracy of the final manuscript.

Conflicts of Interest: The authors have no conflicts of interest to declare. All co-authors have seen and agree with the contents of the manuscript and there is no financial interest to report. We certify that the submission is original work and is not under review at any other publication.

Abbreviations

The following abbreviations are used in this manuscript:

ANN	Artificial Neural Network
AUC	Area Under Curve
CV	Cross-Validation
DDoS	Distributed Denial of Service
DL	Deep Learning
DoS	Denial of Service
DT	Decision Tree
ET	Ensemble Learning
FPR	False-Positive Rate
GRU	Gated Recurrent Units
ICMP	Internet Control Message Protocol
IP	Internet Protocol
IoT	Internet of Things
KB	Kilobyte
kNN	k-Nearest Neighbor
LR	Logistic Regression
LSTM	Long Short Term Memory
ML	Machine Learning
NB	Naive Bayes
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
ROC	Receiver Operating Characteristic
SMOTE	Synthetic Minority Oversampling Technique
TCP	Transmission Control Protocol
TPR	True-Positive Rate
UDP	User Datagram Protocol
XGB	XGBoost

References

1. Statista. IoT Connected Devices Worldwide 2019–2030. Available online: <https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/> (accessed on 22 September 2025).
2. Alomari, D.; Anis, F.; Alabdullatif, M.; Aljamaan, H. A Survey on Botnets Attack Detection Utilizing Machine and Deep Learning Models. In Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering (EASE '23), Oulu, Finland, 14–16 June 2023; Association for Computing Machinery: New York, NY, USA, 2023; pp. 493–498. [\[CrossRef\]](#)
3. Arshad, A.; Jabeen, M.; Ubaid, S.; Raza, A.; Abualigah, L.; Aldiabat, K.; Jia, H. A novel ensemble method for enhancing Internet of Things device security against botnet attacks. *Decis. Anal. J.* **2023**, *8*, 100307. [\[CrossRef\]](#)
4. Rbah, Y.; Mahfoudi, M.; Balboul, Y.; Chetioui, K.; Fattah, M.; Mazer, S.; Elbekkali, M.; Bernoussi, B. A machine learning based intrusions detection for IoT botnet attacks. *AIP Conf. Proc.* **2023**, *2814*, 030012. [\[CrossRef\]](#)
5. Özer, E.; İskefiyeli, M.; Azimjonov, J. Toward lightweight intrusion detection systems using the optimal and efficient feature pairs of the Bot-IoT 2018 dataset. *Int. J. Distrib. Sens. Netw.* **2021**, *17*, 15501477211052202. [\[CrossRef\]](#)
6. Moore, A.W.; Papagiannaki, K. Toward the Accurate Identification of Network Applications. In *Passive and Active Network Measurement*; Dovrolis, C., Ed.; PAM 2005. Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2005; Volume 3431. [\[CrossRef\]](#)
7. Sen, S.; Spatscheck, O.; Wang, D. Accurate, scalable in-network identification of p2p traffic using application signatures. In Proceedings of the 13th international conference on World Wide Web (WWW '04), New York, NY, USA, 17–20 May 2004; Association for Computing Machinery: New York, NY, USA, 2004; pp. 512–521. [\[CrossRef\]](#)
8. Madhukar, A.; Williamson, C. A Longitudinal Study of P2P Traffic Classification. In Proceedings of the 14th IEEE International Symposium on Modeling, Analysis, and Simulation, Monterey, CA, USA, 11–14 September 2006; pp. 179–188. [\[CrossRef\]](#)
9. Azab, A.; Khasawneh, M.; Alrabaee, S.; Choo, K.K.; Sarsour, M. Network traffic classification: Techniques, datasets, and challenges. *Digit. Commun. Netw.* **2022**, *10*, 676–692. [\[CrossRef\]](#)
10. Shi, H.; Li, H.; Zhang, D.; Cheng, C.; Wu, W. Efficient and robust feature extraction and selection for traffic classification. *Comput. Netw.* **2017**, *119*, 1–16. [\[CrossRef\]](#)
11. Che, Y. Machine Learning Based Network Attacks Classification. In Proceedings of the 2023 IEEE 3rd International Conference on Power, Electronics and Computer Applications (ICPECA), Shenyang, China, 29–31 January 2023; pp. 1198–1203. [\[CrossRef\]](#)
12. Jenefer, A.; Sam, S.; Nair, V.; Thomas, B.G.; George, A.S.; Thomas, R.; Sunil, A.D. A Robust Deep Learning-based Approach for Network Traffic Classification using CNNs and RNNs. In Proceedings of the 2023 4th International Conference on Signal Processing and Communication (ICSPC), Coimbatore, India, 23–24 March 2023; pp. 106–110.
13. Alissa, K.; Alyas, T.; Zafar, K.; Abbas, Q.; Tabassum, N.; Sakib, S. Botnet Attack Detection in IoT Using Machine Learning. *Comput. Intell. Neurosci.* **2022**, *2022*, 4515642. [\[CrossRef\]](#)
14. Araujo, A.M.; de Neira, A.B.; Nogueira, M. Autonomous machine learning for early bot detection in the internet of things. *Digit. Commun. Netw.* **2023**, *9*, 1301–1309. [\[CrossRef\]](#)
15. Datasets | ISOT Research Lab. Available online: <https://onlineacademiccommunity.uvic.ca/isot/datasets/> (accessed on 22 September 2025).
16. The CTU-13 Dataset. Stratosphere IPS. Available online: <https://www.stratosphereips.org/datasets-ctu13/> (accessed on 22 September 2025).
17. DDoS 2019 Dataset. Canadian Institute for Cybersecurity. Available online: <https://www.unb.ca/cic/datasets/ddos-2019.html> (accessed on 22 September 2025).
18. Saied, M.; Guirguis, S.; Madbouly, M. IoT Cybersecurity: On the Use of Boosting-Based Approaches for Botnet Detection. *IT Prof.* **2024**, *26*, 45–54. [\[CrossRef\]](#)
19. Fok, K.; Zheng, L.; Watt, K.; Su, L.; Thing, V. Automated Botnet traffic detection via machine learning. In Proceedings of the TENCON 2018—2018 IEEE Region 10 Conference, Jeju, Republic of Korea, 28–31 October 2018.
20. Saied, M.; Guirguis, S.; Madbouly, M. A comparative analysis of using ensemble trees for botnet detection and classification in IoT. *Sci. Rep.* **2023**, *13*, 21632. [\[CrossRef\]](#)
21. Meidan, Y.; Bohadana, M.; Mathov, Y.; Mirsky, Y.; Shabtai, A.; Breitenbacher, D.; Elovici, Y. N-baiot—Network-based detection of iot botnet attacks using deep autoencoders. *IEEE Pervasive Comput.* **2018**, *17*, 12–22. [\[CrossRef\]](#)
22. Sheikh, M.S.; Peng, Y. Procedures, criteria, and machine learning techniques for network traffic classification: A survey. *IEEE Access* **2022**, *10*, 61135–61158. [\[CrossRef\]](#)
23. Koroniotis, N.; Moustafa, N.; Sitnikova, E.; Turnbull, B. Towards the development of realistic botnet dataset in the internet of things for network forensic analytics: Bot-iot dataset. *Future Gener. Comput. Syst.* **2019**, *100*, 779–796. [\[CrossRef\]](#)
24. BotNet Dataset. Available online: <https://paperswithcode.com/dataset/botnet> (accessed on 22 September 2025).
25. Datasets | Canadian Institute for Cybersecurity. Available online: <https://www.unb.ca/cic/datasets/index.html> (accessed on 22 September 2025).

26. Rachmawati, S.M.; Kim, D.S.; Lee, J.M. Machine learning algorithm in network traffic classification. In Proceedings of the 2021 International Conference on Information and Communication Technology Convergence (ICTC), Jeju Island, Republic of Korea, 20–22 October 2021; pp. 1010–1013.
27. Esmaeilifard, R.; Shoaee, Z.; Javidan, R. A lightweight and efficient model for botnet detection in IoT using stacked ensemble learning. *Soft Comput.* **2025**, *29*, 89–101. [\[CrossRef\]](#)
28. Saied, M.; Guirguis, S. Explainable artificial intelligence for botnet detection in internet of things. *Sci. Rep.* **2025**, *15*, 7632. [\[CrossRef\]](#)
29. Rupanetti, D.; Kaabouch, N. Leveraging Machine Learning for Botnet Attack Detection in Edge-Computing Assisted IoT Networks. *arXiv* **2025**, arXiv:2508.01542.
30. Ibrahim, Z.-A.; Razali, R.A.; Ismail, S.A.; Azhar, I.H.K.; Rahim, F.A.; Azilan, A.M.A. A Review of Machine Learning Botnet Detection Techniques based on Network Traffic Log. In Proceedings of the 2022 IEEE International Conference on Computing (ICOCO), Kota Kinabalu, Malaysia, 14–16 November 2022; pp. 204–209. [\[CrossRef\]](#)
31. Rawat, M.; Bedi, A.S.; Singh, B.; Gupta, S.; Singal, G.; Kaur, P. Ensemble-Based Botnet Attack Detection and Classification Using Machine Learning Algorithms on NBaIoT Dataset. In Proceedings of the 2024 IEEE Region 10 Symposium (TENSYP), New Delhi, India, 27–29 September 2024; pp. 1–6.
32. Moustafa, N.; Slay, J. UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set). In Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), Canberra, ACT, Australia, 10–12 November 2015; pp. 1–6. [\[CrossRef\]](#)
33. ChethuHN. Network Intrusion Dataset (CIC-IDS 2017). Kaggle. Available online: <https://www.kaggle.com/datasets/chethuhn/network-intrusion-dataset> (accessed on 22 September 2025).
34. Stratosphere Laboratory IoT-23 Dataset. Available online: <https://www.stratosphereips.org/datasets-iot23> (accessed on 22 September 2025).
35. Al Rawajbeh, M.; Maria Soosai, A.J.; Ramasamy, L.K.; Khan, F. Trustworthy Adaptive AI for Real-Time Intrusion Detection in Industrial IoT Security. *IoT* **2025**, *6*, 53. [\[CrossRef\]](#)
36. Neto, E.C.P.; Dadkhah, S.; Ferreira, R.; Zohourian, A.; Lu, R.; Ghorbani, A.A. CICIOT2023: A Real-Time Dataset and Benchmark for Large-Scale Attacks in IoT Environment. *Sensors* **2023**, *23*, 5941. [\[CrossRef\]](#)
37. Moustafa, N. A new distributed architecture for evaluating AI-based security systems at the edge: Network TON_IoT datasets. *Sustain. Cities Soc.* **2021**, *72*, 102994. [\[CrossRef\]](#)
38. Meidan, Y. N-BaIoT. IEEE Dataport. 2025. Available online: <https://doi.org/10.21227/y9de-qj71> (accessed on 22 September 2025).
39. Pallakonda, A.; Kaliyannan, K.; Sumathi, R.L.; Raj, R.D.A.; Yanamala, R.M.R.; Napoli, C.; Randieri, C. AI-Driven Attack Detection and Cryptographic Privacy Protection for Cyber-Resilient Industrial Control Systems. *IoT* **2025**, *6*, 56. [\[CrossRef\]](#)
40. Ye, J.; Wang, Z.; Yang, J.; Wang, C.; Zhang, C. An LDDoS Attack Detection Method Based on Behavioral Characteristics and Stacking Mechanism. *IoT* **2025**, *6*, 7. [\[CrossRef\]](#)
41. Nuha, H.H.; Mugitama, S.A.; Absa, A.A.; Sutiyo. K-Nearest Neighbors with Third-Order Distance for Flooding Attack Classification in Optical Burst Switching Networks. *IoT* **2025**, *6*, 1. [\[CrossRef\]](#)
42. Ali, M.; Mushtaq, M.F.; Akram, U.; Aray, D.G.; Vergara, M.M.; Karamti, H.; Ashraf, I. Botnet detection in internet of things using stacked ensemble learning model. *Sci. Rep.* **2025**, *15*, 21012. [\[CrossRef\]](#)
43. Mohan, H.G.; Kumar, J.; Nandish, M. Host-Level Botnet Detection via Internet DNS Traffic Analysis Using Neural Networks. *Internet Technol. Lett.* **2025**, *8*, E70101. [\[CrossRef\]](#)
44. Kayyidavazhiyil, A. Combined Tri-Classifiers for IoT Botnet Detection with Tuned Training Weights. *Int. J. Image Graph.* **2025**, *25*, 2550007. [\[CrossRef\]](#)
45. Tikekar, P.C.; Sherekar, S.S.; Kumar, J. An approach for detection of botnet based on machine learning classifier. *SN Comput. Sci.* **2024**, *5*, 300. [\[CrossRef\]](#)
46. Hostiadi, D.P.; Ahmad, T.; Putra, M.A.R.; Pradipta, G.A.; Ayu, P.D.W.; Liandana, M. A New Approach of Botnet Activity Detection Models Using Combination of Univariate and ANOVA Feature Selection Techniques. *Int. J. Intell. Eng. Syst.* **2024**, *17*, 485–502. [\[CrossRef\]](#)
47. Mannikar, R.; Di Troia, F. Enhancing Botnet detection in network security using profile hidden Markov models. *Appl. Sci.* **2024**, *14*, 4019. [\[CrossRef\]](#)
48. Bojarajulu, B.; Tanwar, S. Customized convolutional neural network model for IoT botnet attack detection. *Signal Image Video Process.* **2024**, *18*, 5477–5489. [\[CrossRef\]](#)
49. Saif, S.; Yasmin, N.; Biswas, S. Feature engineering based performance analysis of ML and DL algorithms for Botnet attack detection in IoMT. *Int. J. Syst. Assur. Eng. Manag.* **2023**, *14* (Suppl. 1), 512–522. [\[CrossRef\]](#)
50. Chaganti, R.; Suliman, W.; Ravi, V.; Dua, A. Deep learning approach for SDN-enabled intrusion detection system in IoT networks. *Information* **2023**, *14*, 41. [\[CrossRef\]](#)

51. Sharma, B.; Sharma, L.; Lal, C.; Roy, S. Anomaly based network intrusion detection for IoT attacks using deep learning technique. *Comput. Electr. Eng.* **2023**, *107*, 108626. [[CrossRef](#)]
52. Santhadevi, D.; Janet, B. Stacked deep learning framework for edge-based intelligent threat detection in IoT network. *J. Supercomput.* **2023**, *79*, 12622–12655. [[CrossRef](#)]
53. Cam, N.T.; Trung, N.G. An intelligent approach to improving the performance of threat detection in iot. *IEEE Access* **2023**, *11*, 44319–44334. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.